O Método rDG(P0P1) com HLLC e Minmod

Um Tutorial Didático para o Caso 1D

Leis de conservação hiperbólicas,
volumes finitos, Galerkin Descontínuo reconstruído,
e o fluxo numérico HLLC

Este documento apresenta, passo a passo, o método rDG(P0P1) aplicado às equações de Euler 1D. Cada conceito é motivado intuitivamente, formalizado com equações e ilustrado com código Python e gráficos. Esta versão ampliada detalha as deduções, acrescenta exemplos numéricos resolvidos à mão, um estudo de convergência e figuras novas, geradas por uma implementação de referência que acompanha o texto.

Sumário

1 Leis de conservação e o conceito de fluxo **2**
1.1 O que estamos resolvendo? . . . 2
1.2 Exemplo fundamental: a equação de Burgers . . . 2
1.3 Sistema: as equações de Euler 1D . . . 3

2 Discretização: de contínuo para discreto **3**
2.1 A malha . . . 3
2.2 A média celular e a forma integral . . . 3

3 De volumes finitos a Galerkin Descontínuo (e por que "reconstruído") **4**
3.1 O problema de primeira ordem . . . 4
3.2 A formulação DG(P1) . . . 5
3.3 O caminho "reconstruído": rDG(P0P1) . . . 5

4 Passo 1: reconstrução com minmod **6**
4.1 A inclinação limitada . . . 6
4.2 Implementação . . . 6

5 Passo 2: Estados nas interfaces **7**

6 Passo 3: O fluxo numérico HLLC **8**
6.1 O que é um fluxo numérico? . . . 8
6.2 A ideia do HLLC . . . 8
6.3 A fórmula . . . 9
6.4 Implementação . . . 10

7 Passo 4: Atualização temporal **11**
7.1 O operador espacial . . . 11
7.2 BDF2 (*Backward Differentiation Formula* de 2ª ordem) . . . 11
7.3 Implementação . . . 12

8 O pipeline completo **14**

9 Resultados: tubo de choque de Sod **14**
9.1 Convergência com a malha . . . 15

10 Resumo e perspectivas **15**
10.1 O que aprendemos . . . 15
10.2 Para onde ir . . . 17

1. Leis de conservação e o conceito de fluxo

1.1 O que estamos resolvendo?

Imagine uma quantidade física — por exemplo, a massa de um gás — distribuída ao longo de um tubo unidimensional. Essa quantidade pode se mover (ser *transportada*) ao longo do tubo, mas não pode ser criada nem destruída. Isso é uma **lei de conservação**:

$$\frac{\partial u}{\partial t} + \frac{\partial f(u)}{\partial x} = 0 \tag{1}$$

Aqui:

$u(x,t)$ é a **variável conservativa** (ex.: densidade, momento, energia);

$f(u)$ é o **fluxo**: ele mede *quanto de u passa por um ponto x por unidade de tempo.*

Intuição

Pense em um rio: u é a concentração de um corante na água; $f(u)$ é a "vazão de corante" passando por cada seção do rio. Se entra mais corante do que sai de um trecho, a concentração naquele trecho aumenta — é exatamente isso que a equação (1) diz.

Vale a pena ver essa frase escrita em fórmula, porque é dela que nasce o método numérico. Integre (1) num trecho fixo $[a,b]$ do tubo. O primeiro termo dá a taxa de variação do total de u no trecho; o segundo é a integral de uma derivada, que vale a diferença do fluxo nas pontas:

$$\frac{d}{dt}\int_a^b u(x,t)\,dx = f\big(u(a,t)\big) - f\big(u(b,t)\big). \tag{2}$$

Esta é a **forma integral** da lei de conservação: "o que há dentro só muda pelo que atravessa as fronteiras". Ela tem uma vantagem decisiva sobre (1): não pede que u seja derivável. Quando a solução desenvolve um salto (um choque), a forma diferencial perde o sentido, mas a forma integral continua válida. É por isso que os métodos de volumes finitos partem de (2), não de (1).

Uma última observação sobre (1). Pela regra da cadeia, $\partial_x f(u) = f'(u)\,\partial_x u$, de modo que a equação pode ser lida como $\partial_t u + f'(u)\,\partial_x u = 0$: a informação é transportada com a **velocidade característica** $f'(u)$. Essa velocidade decide tudo o que vem depois: para onde a informação viaja, qual lado da interface "manda" e qual passo de tempo um método explícito poderia usar.

1.2 Exemplo fundamental: a equação de Burgers

O exemplo mais simples e instrutivo de lei de conservação não-linear é a **equação de Burgers inviscida**:

$$\frac{\partial u}{\partial t} + \frac{\partial}{\partial x}\left(\frac{u^2}{2}\right) = 0 \tag{3}$$

Aqui o fluxo é $f(u) = \frac{1}{2}u^2$; a "velocidade de propagação" é $f'(u) = u$: quanto maior o valor de u, mais rápido ele se propaga. Essa não-linearidade faz com que perfis suaves possam *quebrar* e formar **choques** (descontinuidades) em tempo finito — um dos grandes desafios que todo método numérico precisa enfrentar.

Intuição

Imagine carros numa estrada em que cada carro anda com velocidade igual ao seu "valor de u". Os carros rápidos, atrás, alcançam os lentos, na frente; a fila se comprime até virar uma parede: o choque. Para um perfil inicial $u_0(x)$, isso acontece no instante $t_b = -1/\min u_0'(x)$, o momento em que duas características se cruzam pela primeira vez. Depois de t_b a solução suave deixa de existir; só a forma integral (2) dá sentido ao que acontece; ela fixa a velocidade do choque em $s = \frac{1}{2}(u_L + u_R)$, a média dos valores dos dois lados.

1.3 Sistema: as equações de Euler 1D

Para a dinâmica de gases, a lei de conservação é um **sistema** com três componentes:

$$\mathbf{u} = \begin{pmatrix} \rho \\ \rho v \\ E \end{pmatrix}, \qquad \mathbf{f}(\mathbf{u}) = \begin{pmatrix} \rho v \\ \rho v^2 + p \\ (E+p)\,v \end{pmatrix} \tag{4}$$

com a equação de estado do gás ideal:

$$p = (\gamma - 1)\big(E - \tfrac{1}{2}\rho v^2\big) \tag{5}$$

Cada linha é uma conservação: de massa (ρ), de momento (ρv) e de energia (E). O fluxo agora é um vetor que acopla as três quantidades.

Lendo as componentes do fluxo linha a linha fica claro de onde vem cada uma: a massa é transportada com velocidade v (fluxo ρv); o momento é transportado ($\rho v \cdot v$) e, além disso, empurrado pela pressão (p); a energia é transportada (Ev) e a pressão realiza trabalho (pv). A variável E é a energia total por unidade de volume: energia interna ρe mais energia cinética $\frac{1}{2}\rho v^2$. Para o gás ideal, $p = (\gamma-1)\rho e$, o que dá (5); usaremos $\gamma = 1{,}4$ (ar).

Intuição

O sistema de Euler é a "versão séria" de Burgers: em vez de uma única velocidade escalar, temos três tipos de ondas — duas ondas acústicas (se propagando a $v \pm a$, onde $a = \sqrt{\gamma p/\rho}$ é a velocidade do som) e uma onda de contato (se propagando a v). É para resolver essas três ondas corretamente que usaremos o fluxo HLLC.

As três velocidades são os autovalores da matriz jacobiana $\partial\mathbf{f}/\partial\mathbf{u}$; o fato de serem reais e distintas é o que torna o sistema *hiperbólico* e garante que a informação viaja com velocidade finita. A onda de contato é especial: através dela a pressão e a velocidade são contínuas e só a densidade salta, como a fronteira entre dois gases de temperaturas diferentes que se movem juntos. Nenhum mecanismo a comprime, por isso ela é a onda mais difícil de manter nítida numericamente.

2. Discretização: de contínuo para discreto

2.1 A malha

Dividimos o domínio em N células:

$$I_i = [x_{i-1/2},\, x_{i+1/2}], \qquad i = 0, 1, \ldots, N-1$$

com centro x_i e tamanho $\Delta x = x_{i+1/2} - x_{i-1/2}$.

Os pontos $x_{i\pm1/2}$ são as **interfaces** (ou faces) entre células vizinhas; é nelas que toda a comunicação acontece. Para tratar os contornos sem casos especiais, acrescentamos uma **célula fantasma** em cada extremidade e copiamos nela o valor da célula vizinha. Isso implementa a condição *transmissiva*: as ondas saem do domínio como se o tubo continuasse. Para um domínio periódico, a célula fantasma recebe o valor da célula do outro extremo.

2.2 A média celular e a forma integral

A ideia-chave dos volumes finitos é trabalhar com a **média celular**:

$$\bar{u}_i(t) = \frac{1}{\Delta x}\int_{x_{i-1/2}}^{x_{i+1/2}} u(x,t)\,dx \tag{6}$$

Se integrarmos a lei de conservação (1) sobre a célula I_i, obtemos:

$$\frac{d\bar{u}_i}{dt} = -\frac{1}{\Delta x}\Big[\hat{f}_{i+1/2} - \hat{f}_{i-1/2}\Big] \tag{7}$$

onde $\hat{f}_{i+1/2}$ é o **fluxo numérico** na interface entre as células i e $i+1$.

Vale percorrer o caminho de (1) até (7) com calma, porque ele mostra onde entra a única aproximação do método. Aplique a forma integral (2) ao trecho $[a,b] = [x_{i-1/2}, x_{i+1/2}]$ e divida por Δx:

$$\frac{d\bar{u}_i}{dt} = -\frac{1}{\Delta x}\Big[f\big(u(x_{i+1/2},t)\big) - f\big(u(x_{i-1/2},t)\big)\Big].$$

Até aqui *nada* foi aproximado: esta relação é exata para a solução verdadeira. O problema é que o método só conhece as médias $\bar{u}_i$, não os valores pontuais $u(x_{i\pm1/2},t)$ nas interfaces. A aproximação consiste em substituir o fluxo físico na interface por um *fluxo numérico* $\hat{f}_{i+1/2}$, calculado a partir das médias das células vizinhas. Como escolher esse fluxo é o assunto dos Passos 1 a 3.

Intuição

A equação (7) é um **balanço de caixa**: a taxa de variação da média $\bar{u}_i$ é igual ao fluxo que entra pela esquerda ($\hat{f}_{i-1/2}$) menos o fluxo que sai pela direita ($\hat{f}_{i+1/2}$), dividido pelo tamanho da célula. Se entra mais do que sai, $\bar{u}_i$ aumenta; se sai mais, diminui. É exatamente isso que a Figura 5 ilustra. Há uma consequência imediata e preciosa: o fluxo que sai da célula i pela face $x_{i+1/2}$ é o mesmo que entra na célula $i+1$. Somando (7) sobre todas as células, os fluxos interiores cancelam aos pares e o total $\sum_i \bar{u}_i \Delta x$ só muda pelo que atravessa as duas extremidades do domínio. O método é **conservativo por construção**, com ou sem choques; é isso que garante que os choques numéricos andem na velocidade certa.

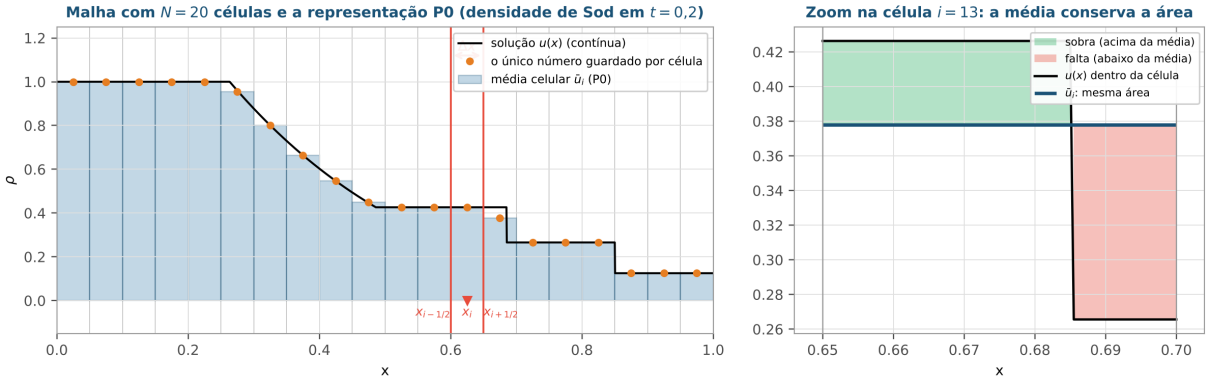


Figura 1: A malha e as médias celulares $\bar{u}_i$. Como exemplo usamos a densidade da solução exata do tubo de choque de Sod em $t = 0{,}2$ (curva preta), que tem uma parte suave e dois saltos. Esquerda: cada célula armazena um único número, a média da solução naquela célula — esta é a representação P0 (constante por partes). Note o que se perde: a rampa suave vira uma escada e os saltos ficam "escondidos" dentro de uma célula. Direita: zoom numa célula que contém o salto; a média é a altura do retângulo que tem a mesma área que a curva, a parte verde (sobra) compensa a vermelha (falta).

3. De volumes finitos a Galerkin Descontínuo (e por que "reconstruído")

3.1 O problema de primeira ordem

O método de volumes finitos descrito acima — com médias constantes por célula — é de **primeira ordem** de precisão. Isso significa que o erro diminui linearmente com Δx: para cortar o erro pela metade, precisamos dobrar o número de células. Na prática, isso gera soluções muito difusas, com choques "borrados" e regiões suaves mal resolvidas.

Intuição

De onde vem a primeira ordem? Ao representar a solução por um valor constante em cada célula, cometemos em cada interface um erro proporcional à inclinação local vezes Δx (a diferença entre o valor na face e a média). O fluxo numérico transforma esse erro num "atrito" numérico: a solução se comporta como se tivesse uma viscosidade artificial proporcional a Δx. É essa viscosidade que borra os choques e achata os picos. Com uma reta por célula, o erro na face cai para a ordem de Δx^2 (a curvatura vezes Δx^2) e a viscosidade artificial desaparece nas regiões suaves.

Para obter **segunda ordem**, precisamos de mais informação dentro de cada célula. Há dois caminhos clássicos:

1. **MUSCL** (Monotone Upstream-centred Schemes for Conservation Laws): mantém apenas as médias $\bar{u}_i$ como incógnitas e **reconstrói** uma inclinação $\hat{u}_i$ a cada passo usando as médias vizinhas.
2. **DG (Galerkin Descontínuo)**: armazena e evolui **ambos** $\bar{u}_i$ e $\hat{u}_i$ como incógnitas independentes, cada uma com sua própria equação de evolução.

3.2 A formulação DG(P1)

No DG, a solução dentro de cada célula é um polinômio:

$$u_h(x)\big|_{I_i} = \bar{u}_i\,\varphi_0(\xi) + \hat{u}_i\,\varphi_1(\xi), \qquad \varphi_0 = 1, \quad \varphi_1 = \xi \tag{8}$$

onde $\xi = \frac{x - x_i}{\Delta x/2} \in [-1, +1]$ é a coordenada local. A formulação fraca (multiplicar por funções-teste φ_k e integrar) produz *duas* equações por célula:

$$\int_{I_i} \frac{\partial u_h}{\partial t}\,\varphi_k\,dx - \int_{I_i} f(u_h)\,\frac{d\varphi_k}{dx}\,dx + \Big[\hat{f}\,\varphi_k\Big]_{x_{i-1/2}}^{x_{i+1/2}} = 0, \qquad k = 0, 1 \tag{9}$$

- $k = 0$ ($\varphi_0 = 1$): dá a equação de volumes finitos para $\bar{u}_i$ — é exatamente (7);
- $k = 1$ ($\varphi_1 = \xi$): dá uma equação *adicional* para evoluir $\hat{u}_i$ no tempo.

Dois detalhes da escolha da base merecem atenção. Primeiro, $\varphi_1 = \xi$ tem média zero na célula ($\int_{-1}^{1} \xi\,d\xi = 0$), logo o coeficiente $\bar{u}_i$ em (8) é *sempre* a média celular, qualquer que seja $\hat{u}_i$: a reta passa pela média no centro da célula. Segundo, $\hat{u}_i$ é a variação da reta do centro até a face: nos extremos $\xi = \pm 1$ a solução vale $\bar{u}_i \pm \hat{u}_i$. A inclinação física é $2\hat{u}_i/\Delta x$. Guardar esses dois significados em mente evita confusões de fatores 2 e $\frac{1}{2}$ mais adiante.

Como (9) aparece: multiplica-se (1) por φ_k, integra-se na célula e integra-se por partes o termo de fluxo. A integração por partes transfere a derivada do fluxo para a função-teste (o termo de volume) e deixa nas faces o fluxo avaliado nos contornos (o termo de fronteira). Como u_h é descontínua nas faces, o fluxo ali precisa ser o fluxo numérico $\hat{f}$, o mesmo do método de volumes finitos. Para $k = 1$ o termo de volume não é nulo e exige quadratura de Gauss; o termo $\int \partial_t u_h\,\varphi_k\,dx$ gera uma pequena *matriz de massa* por célula.

3.3 O caminho "reconstruído": rDG(P0P1)

O DG(P1) é elegante, mas tem um custo: precisa evoluir $2N$ incógnitas em vez de N; além disso, a integral do fluxo com φ_1 é não-trivial (envolve quadratura de Gauss).

O **rDG(P0P1)** (reconstructed DG) é um híbrido:

- **Evolui apenas** as médias $\bar{u}_i$ (equação com $k = 0$, ou seja, FV puro);
- **Reconstrói** a inclinação $\hat{u}_i$ a partir dos vizinhos a cada passo, em vez de evoluir via $k = 1$.

A notação P_nP_m resume a família: guarda-se e evolui-se um polinômio de grau n, mas usa-se nos fluxos um polinômio reconstruído de grau $m \geq n$. Assim rDG(P0P0) é o volume finito de primeira ordem, rDG(P0P1) é o método deste tutorial, rDG(P1P2) guarda a reta e reconstrói a parábola (terceira ordem) e DG(P1) é o caso sem reconstrução, $n = m = 1$. A reconstrução é possível porque a informação "que falta" na célula i está disponível nas células vizinhas; o rDG apenas a recupera no momento do uso, em vez de carregá-la como incógnita.

Intuição

O rDG(P0P1) é essencialmente um método MUSCL, mas formulado na linguagem DG. A vantagem dessa formulação é tripla: (i) a extensão para ordens superiores é sistemática (rDG(P0P2), rDG(P1P2), etc.); (ii) a análise de estabilidade e convergência se apoia na teoria DG; (iii) a implementação se integra naturalmente a frameworks DG existentes.

4. Passo 1: reconstrução com minmod

4.1 A inclinação limitada

Precisamos calcular $\hat{u}_i$ a partir das médias vizinhas. A maneira mais natural é usar diferenças:

$$\Delta_{+i} = \bar{u}_{i+1} - \bar{u}_i \quad \text{(forward)}, \qquad \Delta_{-i} = \bar{u}_i - \bar{u}_{i-1} \quad \text{(backward)} \tag{10}$$

Se ambas concordam em sinal, o perfil é localmente monótono e podemos usar a menor das duas (em valor absoluto) como inclinação. Se discordam, estamos num extremo local ou choque; a inclinação deve ser zero. Isso é o **limitador minmod**:

$$\hat{u}_i = \frac{1}{2}\,\mathrm{minmod}(\Delta_{+i}, \Delta_{-i}) \equiv \frac{1}{2}\begin{cases} \mathrm{sgn}(\Delta_{+i})\,\min(|\Delta_{+i}|, |\Delta_{-i}|) & \text{se } \Delta_{+i}\cdot\Delta_{-i} > 0 \\ 0 & \text{caso contrário} \end{cases} \tag{11}$$

De onde vem o fator $\frac{1}{2}$? Se a solução fosse exatamente uma reta, teríamos $\Delta_{+i} = \Delta_{-i} = \Delta$ e a reta subiria Δ de um centro de célula ao seguinte, ou seja, $\Delta/2$ do centro até a face. Como $\hat{u}_i$ é justamente a variação do centro até a face (Seção 3.2), a escolha $\hat{u}_i = \Delta/2$ reproduz a reta sem erro. Sem o limitador, a opção natural seria a média das duas diferenças, $\hat{u}_i = \frac{1}{4}(\Delta_{+i} + \Delta_{-i}) = \frac{1}{4}(\bar{u}_{i+1} - \bar{u}_{i-1})$, a boa e velha diferença central. O minmod troca a média pela menor das duas, em módulo: é a versão "cautelosa" da diferença central.

Um exemplo à mão. Tome três médias consecutivas $\bar{u}_{i-1} = 0{,}20$, $\bar{u}_i = 0{,}50$ e $\bar{u}_{i+1} = 0{,}60$. Então $\Delta_{-i} = 0{,}30$ e $\Delta_{+i} = 0{,}10$: mesmo sinal, logo $\hat{u}_i = \frac{1}{2}\cdot 0{,}10 = 0{,}05$ e os valores de face são 0,45 e 0,55, ambos dentro do intervalo dos vizinhos. A diferença central daria $\hat{u}_i = 0{,}10$ e o valor de face 0,60, igual ao vizinho da direita: uma inclinação no limite do aceitável. Agora troque $\bar{u}_{i+1}$ por 0,40: $\Delta_{+i} = -0{,}10$ discorda de $\Delta_{-i} = +0{,}30$, a célula é um máximo local e $\hat{u}_i = 0$.

Intuição

(a) Quando os dois vizinhos concordam (mesma direção de variação), o minmod escolhe a inclinação **mais conservadora** (menor em módulo). Isso evita que a reconstrução crie "overshoots" além dos dados: os valores de face ficam sempre entre a média da célula e a média do vizinho correspondente.
(b) Quando os vizinhos discordam (um cresce e outro decresce: extremo local ou choque), ele **zera a inclinação**. Nesse caso, a célula degenera para P0 (constante); o método cai para primeira ordem *localmente*. Essa degeneração é proposital: é o preço que pagamos para evitar oscilações espúrias (fenômeno de Gibbs) perto de descontinuidades.
O nome técnico dessa propriedade é TVD (*total variation diminishing*): a soma dos saltos $\sum_i |\bar{u}_{i+1} - \bar{u}_i|$ nunca aumenta de um passo para o outro, logo nenhuma oscilação nova pode nascer. Há um teorema de Godunov dizendo que nenhum esquema *linear* consegue ser ao mesmo tempo de segunda ordem e livre de oscilações; o limitador é a saída não linear, porque a inclinação depende dos próprios dados.

4.2 Implementação

Listing 1: Reconstrução minmod (Python). As células fantasma copiam o valor da vizinha (contorno transmissivo); U tem uma linha por variável conservada e uma coluna por célula.

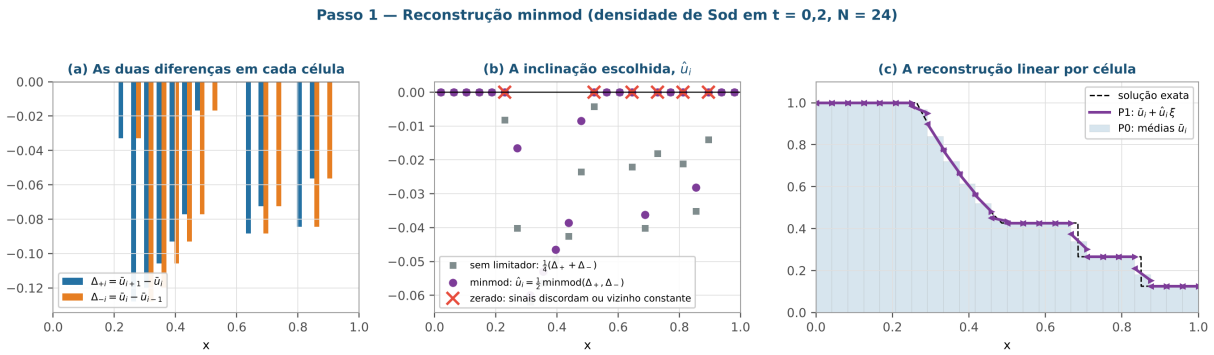


Figura 2: **Passo 1 em ação**, sobre as médias da densidade de Sod em $t = 0{,}2$ (24 células). (a) As duas diferenças de cada célula, Δ_{+i} (azul) e Δ_{-i} (laranja); nos trechos constantes elas são nulas. (b) A inclinação escolhida: o minmod (círculos roxos) fica sempre com a menor das duas em módulo, enquanto a diferença central (quadrados cinza) fica com a média; as cruzes vermelhas marcam as células em que o minmod zerou a inclinação, porque um dos vizinhos é constante ou os sinais discordam, como nas duas células que contêm os saltos. (c) A reconstrução linear resultante (roxo) sobre as médias (azul-claro) e a solução exata (tracejada): nas regiões suaves a reta acompanha a curva; nas células dos saltos ela é horizontal.

```python
def minmod(a, b):
    """Menor dos dois em modulo se os sinais coincidem; zero caso contrario."""
    same_sign = (a * b > 0)
    return np.where(same_sign, np.where(np.abs(a) < np.abs(b), a, b), 0.0)

def ghost(U):
    """Acrescenta uma celula fantasma de cada lado (copia da vizinha)."""
    return np.concatenate((U[:, :1], U, U[:, -1:]), axis=1)

def reconstruct(U):
    """Coeficiente u_hat_i de cada celula: valor de face = media +- u_hat."""
    Ug = ghost(U)
    delta_plus  = Ug[:, 2:]   - Ug[:, 1:-1]      # Delta_+ = u_{i+1} - u_i
    delta_minus = Ug[:, 1:-1] - Ug[:, :-2]       # Delta_- = u_i - u_{i-1}
    return 0.5 * minmod(delta_plus, delta_minus)
```

A reconstrução é feita *componente a componente* nas variáveis conservadas (ρ, ρv, E). É a escolha mais simples e a usada neste tutorial; variantes reconstroem nas variáveis primitivas (ρ, v, p) ou nas características, com resultados um pouco mais nítidos ao custo de mais álgebra.

5. Passo 2: Estados nas interfaces

Com a reconstrução feita, avaliamos u_h nos dois lados de cada interface. Na interface $x_{i+1/2}$:

$$u^-_{i+1/2} = \bar{u}_i + \hat{u}_i \qquad \text{(limite pela esquerda, } \xi = +1) \tag{12}$$

$$u^+_{i+1/2} = \bar{u}_{i+1} - \hat{u}_{i+1} \qquad \text{(limite pela direita, } \xi = -1) \tag{13}$$

Em geral, $u^- \neq u^+$ — a solução é **descontínua** na interface. Isso é uma característica central do framework DG: cada célula "não conversa" com a vizinha diretamente. A comunicação acontece via o **fluxo numérico**, que resolve o "conflito" entre os dois valores.

Continuando o exemplo à mão da Seção 4.1: com $\bar{u}_i = 0{,}50$ e $\hat{u}_i = 0{,}05$, o lado esquerdo da face $x_{i+1/2}$ vale $u^- = 0{,}55$. Se a célula seguinte tem $\bar{u}_{i+1} = 0{,}60$ e, digamos, $\hat{u}_{i+1} = 0{,}03$, o lado direito vale $u^+ = 0{,}57$. O salto na face é 0,02, bem menor que o salto 0,10 entre as médias. Essa é a diferença entre primeira e segunda ordem numa frase: **a reconstrução encolhe os saltos nas faces onde a solução é suave**; o fluxo numérico, que "vê" apenas o salto, passa a trabalhar quase como se a solução fosse contínua.

Intuição

Os dois valores u^- e u^+ definem um **problema de Riemann local** na interface — exatamente o tipo de problema que Euler, Burgers e todas as leis de conservação hiperbólicas geram: dado um salto em u, o que acontece?
Na equação de Burgers, se $u^- > u^+$, forma-se um choque; se $u^- < u^+$, uma rarefação. No sistema de Euler, a situação é mais rica: podem surgir até três ondas simultaneamente.

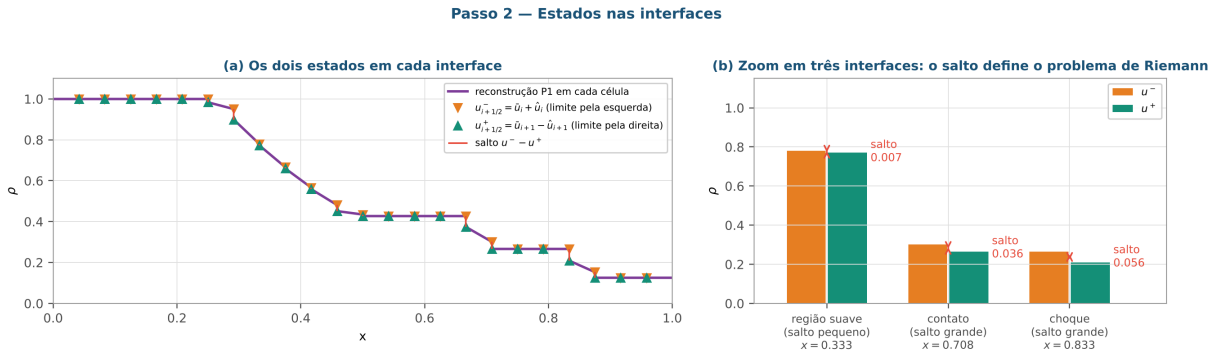


Figura 3: **Passo 2.** Esquerda: a reconstrução P1 de cada célula (roxo) e os dois estados em cada interface, u^- (triângulo laranja, vindo da célula da esquerda) e u^+ (triângulo verde, vindo da célula da direita); o segmento vermelho entre eles é o salto. Onde a solução é suave os dois triângulos quase coincidem; nas células do contato e do choque, em que o minmod zerou a inclinação, o salto é grande. Direita: zoom em três interfaces com os valores numéricos dos saltos; cada par (u^-, u^+) é o dado de um problema de Riemann local.

6. Passo 3: O fluxo numérico HLLC

6.1 O que é um fluxo numérico?

O fluxo numérico $\hat{f}_{i+1/2} = \hat{f}(u^-_{i+1/2}, u^+_{i+1/2})$ é uma função que, dados os dois estados na interface, retorna um *único* valor de fluxo para usar em (7). Ele é o coração do método: é ele que determina quanta "coisa" passa de uma célula para a outra.

Propriedades essenciais:

- **Consistência**: $\hat{f}(u, u) = f(u)$ (se não há salto, o fluxo numérico coincide com o físico);
- **Upwinding**: a informação deve viajar na direção correta (respeitando as características).

A maneira mais fiel de obter esse valor é resolver exatamente o problema de Riemann da interface e ler a solução sobre a própria interface ($x/t = 0$): é o **método de Godunov**. Para Euler isso custa uma iteração de Newton por interface e por passo. Os *solvers aproximados*, como o HLLC, dão uma resposta quase tão boa com álgebra fechada. Todos eles podem ser escritos como "média dos dois fluxos menos uma dissipação proporcional ao salto": a arte está em dosar a dissipação de cada onda.

6.2 A ideia do HLLC

O HLLC (Harten–Lax–van Leer–Contact) é um **solver de Riemann aproximado** para sistemas. Ele assume que a solução do problema de Riemann local tem uma estrutura simplificada com **três ondas**:

$$\mathbf{u}_L \xrightarrow{S_L} \mathbf{u}_{*L} \xrightarrow{S_*} \mathbf{u}_{*R} \xrightarrow{S_R} \mathbf{u}_R$$

- S_L, S_R: ondas externas (acústicas, no caso de Euler);
- S_*: onda de contato (a "Contact" do HLLC) — separa duas regiões que têm a mesma pressão e velocidade S_*, mas densidades diferentes.

Entre as ondas, a solução é suposta *constante*: dois estados "estrela", $\mathbf{u}_{*L}$ e $\mathbf{u}_{*R}$. Eles não são chutados: saem das relações de salto (Rankine–Hugoniot) aplicadas através de S_L e de S_R, com a exigência de que velocidade e pressão sejam as mesmas dos dois lados do contato. O resultado, para $K = L, R$, é

$$\mathbf{u}_{*K} = \rho_K \frac{S_K - v_K}{S_K - S_*} \begin{pmatrix} 1 \\ S_* \\ \dfrac{E_K}{\rho_K} + (S_* - v_K)\left(S_* + \dfrac{p_K}{\rho_K (S_K - v_K)}\right) \end{pmatrix}. \tag{14}$$

O fator $\rho_K(S_K - v_K)/(S_K - S_*)$ é a conservação de massa através da onda S_K escrita no referencial da onda: o que entra por um lado com velocidade relativa $S_K - v_K$ sai pelo outro com velocidade relativa $S_K - S_*$.

Intuição

O HLL original (sem o "C") assume apenas duas ondas e um único estado intermediário. Isso é suficiente para capturar choques e rarefações, mas *borra completamente* as descontinuidades de contato (interfaces entre materiais, por exemplo). O HLLC adiciona a onda de contato, resolvendo esse problema.
O motivo é simples: num contato só a densidade salta, com p e v contínuos. O HLL mistura os dois lados num estado médio único e trata esse salto como se fosse uma onda acústica, dissipando-o com a velocidade do som. O HLLC coloca o salto de densidade na onda S_*, que viaja com o fluido, dissipando-o só com $|S_*|$: numa frente de contato lenta, a diferença entre borrar e não borrar.

6.3 A fórmula

O fluxo HLLC depende da posição relativa da interface em relação às três ondas:

$$\hat{f}_{i+1/2} = \begin{cases} \mathbf{f}_L & \text{se } S_L > 0 \\ \mathbf{f}_{*L} = \mathbf{f}_L + S_L(\mathbf{u}_{*L} - \mathbf{u}_L) & \text{se } S_L \le 0 < S_* \\ \mathbf{f}_{*R} = \mathbf{f}_R + S_R(\mathbf{u}_{*R} - \mathbf{u}_R) & \text{se } S_* \le 0 \le S_R \\ \mathbf{f}_R & \text{se } S_R < 0 \end{cases} \tag{15}$$

As velocidades de onda são estimadas via Davis:

$$S_L = \min(v_L - a_L,\ v_R - a_R), \qquad S_R = \max(v_L + a_L,\ v_R + a_R) \tag{16}$$

e a velocidade de contato:

$$S_* = \frac{p_R - p_L + \rho_L v_L(S_L - v_L) - \rho_R v_R(S_R - v_R)}{\rho_L(S_L - v_L) - \rho_R(S_R - v_R)} \tag{17}$$

Por que as fórmulas dos ramos intermediários têm a forma $\mathbf{f}_K + S_K(\mathbf{u}_{*K} - \mathbf{u}_K)$? É de novo Rankine–Hugoniot: através de uma onda de velocidade S, o salto de fluxo é S vezes o salto de estado. Partindo de $\mathbf{f}_L$ e atravessando a onda S_L chega-se ao fluxo da região estrela esquerda. A estimativa de Davis (16) escolhe para S_L e S_R a menor e a maior velocidade acústica dos dois estados: com isso as ondas externas do modelo são pelo menos tão rápidas quanto as verdadeiras, o que garante a dissipação necessária para que nenhum estado não físico apareça. A fórmula (17) é o que resulta de impor que o momento se conserve entre S_L e S_R com $v = S_*$ e p iguais nas duas regiões estrela.

Intuição

Os quatro ramos de (15) correspondem a quatro situações físicas: (i) todas as ondas vão para a direita (escoamento supersônico para a direita); (ii) a interface está entre S_L e S_* (região estrela esquerda); (iii) a interface está entre S_* e S_R (região estrela direita); (iv) todas as ondas vão para a esquerda.
A pergunta que a fórmula responde é sempre a mesma: "olhando a interface, $x/t = 0$, no diagrama de ondas, em qual das quatro regiões ela está?" O fluxo numérico é o fluxo da região em que a interface cai. Na interface central do tubo de Sod no instante inicial, $S_L = -1{,}18$, $S_* = 0{,}68$ e $S_R = 1{,}18$: a interface está na região estrela esquerda e vale o ramo (ii).

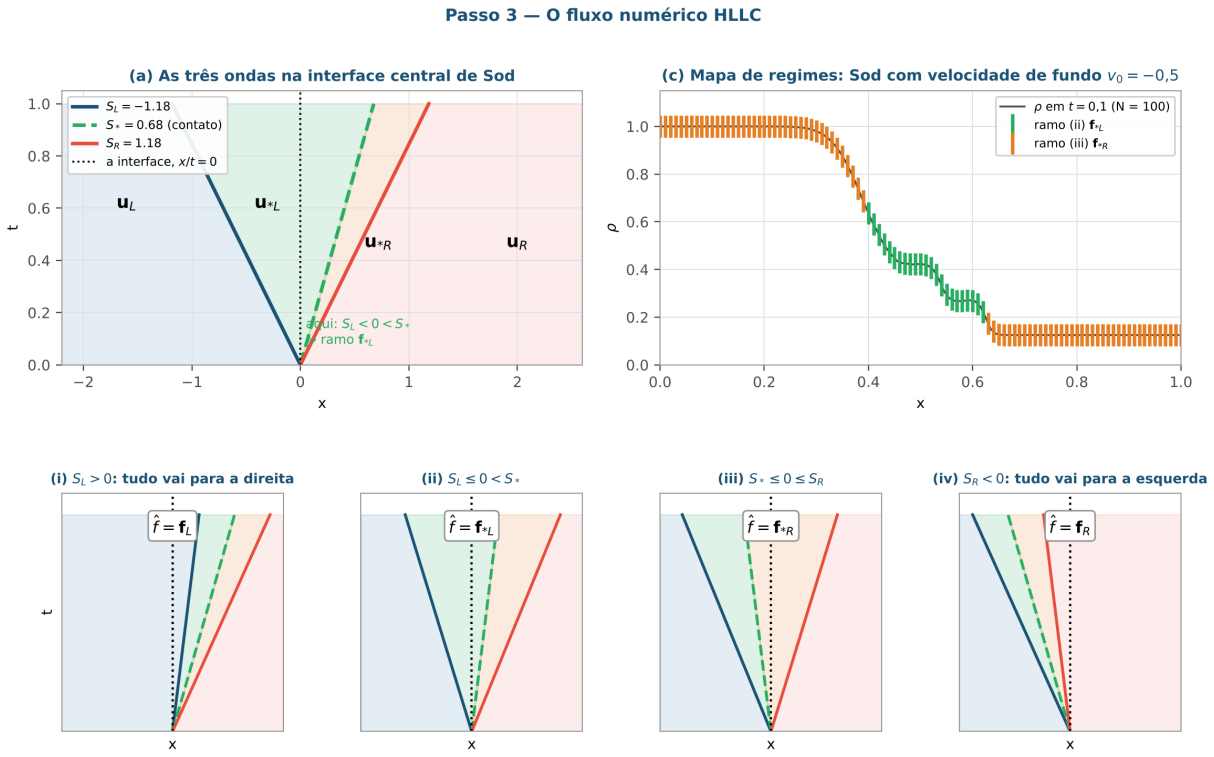


Figura 4: **Passo 3.** (a) Diagrama espaço-tempo das três ondas do HLLC para a interface central do tubo de Sod no instante inicial; as quatro regiões coloridas são os quatro estados; a linha pontilhada é a interface. (b) Os quatro ramos de (15): o que muda de um para outro é a região em que a interface cai. (c) Mapa de regimes numa simulação real: o tubo de Sod com uma velocidade de fundo $v_0 = -0{,}5$ em $t = 0{,}1$; cada traço vertical marca uma interface, colorida pelo ramo usado. Onde o gás anda para a esquerda a interface cai na região estrela direita (laranja, ramo iii); onde a rarefação o acelera para a direita, na região estrela esquerda (verde, ramo ii). Sem velocidade de fundo todas as interfaces usariam o ramo (ii), por isso o mapa do problema de Sod puro é pouco instrutivo.

6.4 Implementação

Listing 2: Fluxo HLLC vetorizado (trechos-chave). As funções conserved_to_primitive, sound_speed e euler_flux implementam (5) e (4).

```
def hllc_flux(uL, uR):
    rhoL, vL, pL = conserved_to_primitive(uL)
    rhoR, vR, pR = conserved_to_primitive(uR)
    aL, aR = sound_speed(rhoL, pL), sound_speed(rhoR, pR)
    fL, fR = euler_flux(uL), euler_flux(uR)

    # Velocidades de onda (Davis)
    SL = np.minimum(vL - aL, vR - aR)
    SR = np.maximum(vL + aL, vR + aR)

    # Onda de contato
    num = pR - pL + rhoL*vL*(SL-vL) - rhoR*vR*(SR-vR)
    den = rhoL*(SL - vL) - rhoR*(SR - vR)
    S_star = num / den

    # Estados estrela, eq. (14)
    def star(u, rho, v, p, S):
        fac = rho * (S - v) / (S - S_star)
        return np.array([fac, fac*S_star,
                         fac*(u[2]/rho + (S_star - v)*(S_star + p/(rho*(S - v))))])
    u_starL = star(uL, rhoL, vL, pL, SL)
    u_starR = star(uR, rhoR, vR, pR, SR)
```

```
23
24      # Montar fluxo por regiao
25      F = np.where(SL >= 0, fL,
26          np.where(S_star >= 0,
27                   fL + SL*(u_starL - uL),       # regiao *L
28          np.where(SR >= 0,
29                   fR + SR*(u_starR - uR),       # regiao *R
30                   fR)))                         # regiao R
31      return F
```

7. Passo 4: Atualização temporal

7.1 O operador espacial

Juntando os passos anteriores, definimos o operador espacial:

$$L(\bar{u})_i = -\frac{1}{\Delta x}\left[\hat{f}_{i+1/2} - \hat{f}_{i-1/2}\right] \tag{18}$$

de modo que $\frac{d\bar{u}_i}{dt} = L(\bar{u})_i$. Cada avaliação de L envolve o pipeline completo: reconstrução $\rightarrow$ interfaces $\rightarrow$ HLLC.

O que sobrou é um sistema de $3N$ equações diferenciais ordinárias no tempo, uma por variável conservada e por célula: é a abordagem conhecida como *método das linhas*. Qualquer integrador de EDOs serve em princípio; a escolha entre explícito e implícito é a decisão que resta.

7.2 BDF2 (*Backward Differentiation Formula* de 2ª ordem)

Para integrar no tempo, utilizamos o método BDF2 (totalmente implícito), que aproxima a derivada temporal em t^{n+1} usando os dois instantes anteriores:

$$\frac{3\,\bar{u}^{n+1} - 4\,\bar{u}^{n} + \bar{u}^{n-1}}{2\,\Delta t} = L(\bar{u}^{n+1}) \tag{19}$$

A fórmula vem de um argumento de interpolação: faça passar uma parábola pelos três valores $\bar{u}^{n-1}$, $\bar{u}^{n}$ e $\bar{u}^{n+1}$ (igualmente espaçados em Δt) e derive-a no último ponto. O resultado é o lado esquerdo de (19); os pesos 3, -4 e 1 são os da derivada dessa parábola. Como a parábola reproduz exatamente qualquer polinômio de grau 2, o erro é $O(\Delta t^2)$: segunda ordem no tempo, compatível com a segunda ordem no espaço.

Rearranjando, o problema a cada passo de tempo é encontrar $\bar{u}^{n+1}$ tal que o **resíduo** se anule:

$$\mathbf{F}(\bar{u}^{n+1}) \equiv 3\,\bar{u}^{n+1} - 4\,\bar{u}^{n} + \bar{u}^{n-1} - 2\,\Delta t\,L(\bar{u}^{n+1}) = \mathbf{0} \tag{20}$$

Este é um sistema **não-linear** (pois L depende de $\bar{u}^{n+1}$ via reconstrução + HLLC) e é resolvido iterativamente pelo método de Newton:

$$\mathbb{J}^{(k)}\,\delta\bar{u} = -\mathbf{F}(\bar{u}^{(k)}), \qquad \bar{u}^{(k+1)} = \bar{u}^{(k)} + \delta\bar{u} \tag{21}$$

onde $\mathbb{J}^{(k)} = \partial\mathbf{F}/\partial\bar{u}\,\big|^{(k)}$ é a matriz Jacobiana. Na prática, pode-se utilizar o método *Jacobian-Free Newton–Krylov* (JFNK) para evitar a montagem explícita de $\mathbb{J}$.

A ideia do JFNK cabe em uma linha. O método de Krylov (GMRES) que resolve o sistema linear (21) nunca pede a matriz $\mathbb{J}$ inteira: pede apenas produtos $\mathbb{J}\mathbf{w}$ para vetores $\mathbf{w}$ que ele próprio escolhe. E um produto Jacobiana-vetor é uma derivada direcional, que se aproxima por diferença finita com *uma* avaliação extra do resíduo:

$$\mathbb{J}\,\mathbf{w} \approx \frac{\mathbf{F}(\bar{u} + \epsilon\,\mathbf{w}) - \mathbf{F}(\bar{u})}{\epsilon}, \qquad \epsilon \sim 10^{-7}. \tag{22}$$

Assim a única coisa que o programador escreve é a função resíduo; a Jacobiana, com os seus $(3N)^2$ coeficientes e as suas derivadas do minmod e do HLLC, nunca é montada.

Atenção

Inicialização: No primeiro passo de tempo ($n = 0 \to 1$), a solução $\bar{u}^{-1}$ não existe. Usa-se então o *Backward Euler* (BDF1, primeira ordem):

$$\frac{\bar{u}^1 - \bar{u}^0}{\Delta t} = L(\bar{u}^1)$$

A partir de $n \geq 1$, o BDF2 é utilizado normalmente.
Passo constante: os pesos 3, -4 e 1 valem para Δt constante. Se o passo mudar de um nível para outro, os coeficientes mudam (BDF2 de passo variável). Neste tutorial fixamos Δt a partir do estado inicial e ajustamos o último passo para terminar exatamente em t_{final}.

Intuição

O BDF2 é um método **implícito** e **A-estável**, o que significa que não há restrição de CFL para estabilidade — o passo de tempo pode ser escolhido com base na *precisão* desejada, não na estabilidade. Isso é essencial em problemas com escalas de tempo díspares (por exemplo, ondas acústicas rápidas e transporte lento), onde métodos explícitos exigiriam passos de tempo proibitivamente pequenos.
O preço a pagar é a necessidade de resolver um sistema não-linear a cada passo de tempo, o que torna cada passo mais caro do que num método explícito. Porém, a possibilidade de usar Δt muito maiores frequentemente compensa esse custo.

A escolha do passo de tempo é guiada pela precisão temporal desejada. Uma regra prática é:

$$\Delta t = \text{CFL} \cdot \frac{\Delta x}{\max_i(|v_i| + a_i)} \tag{23}$$

com CFL tipicamente entre 1 e 100 (dependendo da rigidez do problema), em contraste com o CFL ≤ 1 dos métodos explícitos.

Atenção

Estável não quer dizer preciso. Com CFL $= 5$ o método não explode, mas a informação atravessa cinco células por passo e o BDF2 espalha choques e contatos por várias células, como a Figura 7(c) mostra. Para um problema dominado por ondas rápidas, como o tubo de Sod, o passo grande desperdiça a resolução espacial; é nos transientes lentos, em que as ondas acústicas não interessam, que o CFL alto compensa. Nos resultados deste tutorial usamos CFL $= 1$.
O "Newton simplificado" do Listing 3 só converge com passos pequenos. Trocar a Jacobiana por $3\mathbb{I}$ transforma (21) numa iteração de ponto fixo, $\bar{u} \leftarrow \bar{u} - \mathbf{F}/3$, que contrai apenas se $\frac{2}{3}\Delta t\,\|\partial L/\partial \bar{u}\| < 1$. Como $\|\partial L/\partial \bar{u}\| \approx 2\max(|v| + a)/\Delta x$, a condição é CFL $< 3/4$. Com CFL $= 5$ a iteração diverge em poucos passos (Figura 5c); com CFL $= 0{,}5$ ela converge devagar. O JFNK converge em meia dúzia de iterações nos dois casos. Em resumo: o ponto fixo serve para entender a estrutura do código, o JFNK é o que se usa.

7.3 Implementação

Listing 3: Avanço temporal BDF2 com iterações de Newton (Python), versão mínima. Lê-se bem, mas a aproximação $\mathbb{J} \approx 3\mathbb{I}$ só converge para CFL $< 3/4$ (veja a caixa de atenção acima).

```python
def bdf2_step(U_n, U_nm1, dx, dt, tol=1e-10, max_iter=50):
    """Um passo BDF2: resolve 3*U - 4*U_n + U_nm1 - 2*dt*L(U) = 0."""
    U = U_n.copy()          # chute inicial
    for k in range(max_iter):
        L = compute_rhs(U, dx)
        F = 3*U - 4*U_n + U_nm1 - 2*dt*L      # residuo
        if np.max(np.abs(F)) < tol:
            break
        # Jacobiana aproximada (diagonal): dF/dU ~ 3*I  -> ponto fixo
```

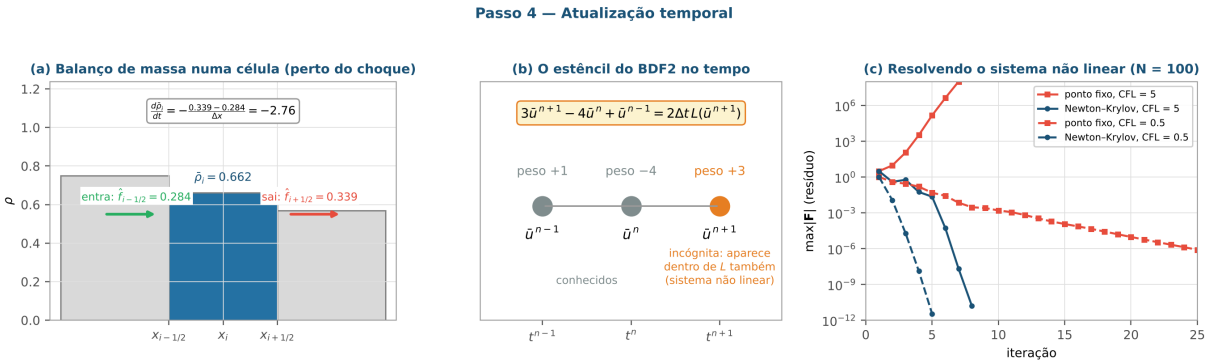


Figura 5: **Passo 4.** (a) Balanço de massa numa célula logo à esquerda do choque de Sod ($t = 0{,}1$, $N = 50$): entra $\hat{f}_{i-1/2}$ pela esquerda, sai $\hat{f}_{i+1/2}$ pela direita e a diferença, dividida por Δx, é a taxa de variação da média — aqui negativa, porque o choque está deixando a célula para trás. (b) O estêncil do BDF2: dois níveis conhecidos, com pesos -4 e $+1$, mais a incógnita $\bar{u}^{n+1}$ com peso 3, que aparece também dentro de L: daí o sistema não linear. (c) Resíduo por iteração no primeiro passo BDF2 ($N = 100$): o Newton–Krylov converge até 10^{-10} em menos de dez iterações para CFL $= 0{,}5$ e CFL $= 5$; a iteração de ponto fixo converge devagar com CFL $= 0{,}5$ e diverge com CFL $= 5$.

```
        dU = -F / 3.0
        U = U + dU
    return U

def backward_euler_step(U_n, dx, dt, tol=1e-10, max_iter=50):
    """Primeiro passo (BDF1): resolve U - U_n - dt*L(U) = 0."""
    U = U_n.copy()
    for k in range(max_iter):
        L = compute_rhs(U, dx)
        F = U - U_n - dt*L
        if np.max(np.abs(F)) < tol:
            break
        dU = -F / 1.0
        U = U + dU
    return U
```

Listing 4: A mesma etapa com Newton–Krylov sem Jacobiana (JFNK). O produto Jacobiana-vetor é a diferença finita (22); o GMRES resolve o sistema linear de cada iteração de Newton de forma aproximada (resíduo linear relativo 10^{-3} basta).

```
from scipy.sparse.linalg import LinearOperator, gmres

def newton_krylov_step(U_n, U_nm1, dx, dt, tol=1e-10, max_iter=50):
    """Resolve F(U) = 0 (BDF2, ou BDF1 se U_nm1 is None) por JFNK."""
    shape = U_n.shape
    def residual(x):
        U = x.reshape(shape)
        L = compute_rhs(U, dx)
        if U_nm1 is None:
            return (U - U_n - dt*L).ravel()
        return (3*U - 4*U_n + U_nm1 - 2*dt*L).ravel()
    x = U_n.ravel().copy()
    for k in range(max_iter):
        F = residual(x)
        if np.max(np.abs(F)) < tol:
            break
        eps = 1e-7 * max(1.0, np.linalg.norm(x))
        def Jv(w):                          # produto Jacobiana-vetor por diferenca finita
            nw = np.linalg.norm(w)
            if nw == 0.0:
                return np.zeros_like(w)
            return (residual(x + (eps/nw)*w) - F) / (eps/nw)
```

```
        J = LinearOperator((x.size, x.size), matvec=Jv, dtype=float)
        dx_newton, info = gmres(J, -F, rtol=1e-3, restart=60, maxiter=200)
        x = x + dx_newton
    return x.reshape(shape)
```

8. O pipeline completo

Resumindo, para **cada iteração de Newton** dentro de um passo BDF2:

Passo	O que faz	Onde está ilustrado
① Reconstrução	$\hat{u}_i = \frac{1}{2}\,\mathrm{minmod}(\Delta_+, \Delta_-)$	Figura 2: retas roxas (P1)
② Interfaces	$u^- = \bar{u}_i + \hat{u}_i,\ \ u^+ = \bar{u}_{i+1} - \hat{u}_{i+1}$	Figura 3: triângulos u^-, u^+
③ Fluxo HLLC	$\hat{f} = \mathrm{HLLC}(u^-, u^+)$	Figura 4: ramos e regimes
④ Atualização BDF2	$3\bar{u}^{n+1} - 4\bar{u}^n + \bar{u}^{n-1} = 2\Delta t\, L(\bar{u}^{n+1})$	Figura 5: balanço e estêncil

Os passos ①–③ são executados a cada avaliação do operador $L(\bar{u})$, ou seja, em cada iteração de Newton do passo ④. A reconstrução minmod é refeita com os dados mais recentes da iteração corrente. Num passo com oito iterações de Newton e, digamos, vinte produtos Jacobiana-vetor por iteração, o operador L é avaliado perto de duzentas vezes: é o custo do implícito, pago em troca da liberdade no passo de tempo.

Listing 5: O pipeline completo em Python. O passo de tempo é fixado uma vez, a partir do estado inicial; é ajustado para terminar exatamente em t_final.

```
def compute_rhs(U, dx):
    """Operador espacial L(u): reconstrucao -> interfaces -> HLLC."""
    S = reconstruct(U)                          # Passo 1
    uL, uR = interface_states(U, S)             # Passo 2 (inclui as faces de contorno)
    F = hllc_flux(uL, uR)                       # Passo 3
    return -(F[:, 1:] - F[:, :-1]) / dx         # uma face a mais que celulas

def time_advance(U, dx, t_final, CFL=1.0):
    """Avanco temporal: BDF1 no primeiro passo, BDF2 nos demais, passo constante."""
    rho, v, p = conserved_to_primitive(U)
    dt = CFL * dx / np.max(np.abs(v) + sound_speed(rho, p))
    n_steps = int(np.ceil(t_final / dt))
    dt = t_final / n_steps                      # termina exatamente em t_final
    U_nm1 = None                                # nao existe no primeiro passo
    for n in range(n_steps):
        U_new = newton_krylov_step(U, U_nm1, dx, dt)   # BDF1 se U_nm1 is None
        U_nm1, U = U, U_new
    return U
```

9. Resultados: tubo de choque de Sod

O teste canônico para métodos de captura de choque é o **tubo de choque de Sod**: um tubo com gás em alta pressão à esquerda e baixa pressão à direita, separados por um diafragma em $x = 0{,}5$:

$$(\rho, v, p) = \begin{cases} (1{,}0,\ 0,\ 1{,}0) & x < 0{,}5 \\ (0{,}125,\ 0,\ 0{,}1) & x \geq 0{,}5 \end{cases}$$

A solução exata em $t = 0{,}2$ contém três estruturas distintas:

1. Uma **onda de rarefação** (suave) propagando-se para a esquerda — aqui o método opera em segunda ordem;
2. Uma **descontinuidade de contato** (salto em ρ mas não em p) — é onde o "C" do HLLC faz diferença;

3. Um **choque** propagando-se para a direita — aqui o minmod degenera para P0 e o método captura o choque sem oscilações.

A solução exata é conhecida: a pressão e a velocidade entre as ondas valem $p^* = 0{,}303$ e $v^* = 0{,}927$, a cabeça da rarefação viaja a $-1{,}18$, o contato a 0,93 e o choque a 1,75. Como o choque é a onda mais rápida, em $t = 0{,}2$ ele está em $x = 0{,}85$, o contato em $x = 0{,}69$ e a rarefação ocupa $0{,}26 < x < 0{,}49$. É com essa solução que comparamos os resultados numéricos.

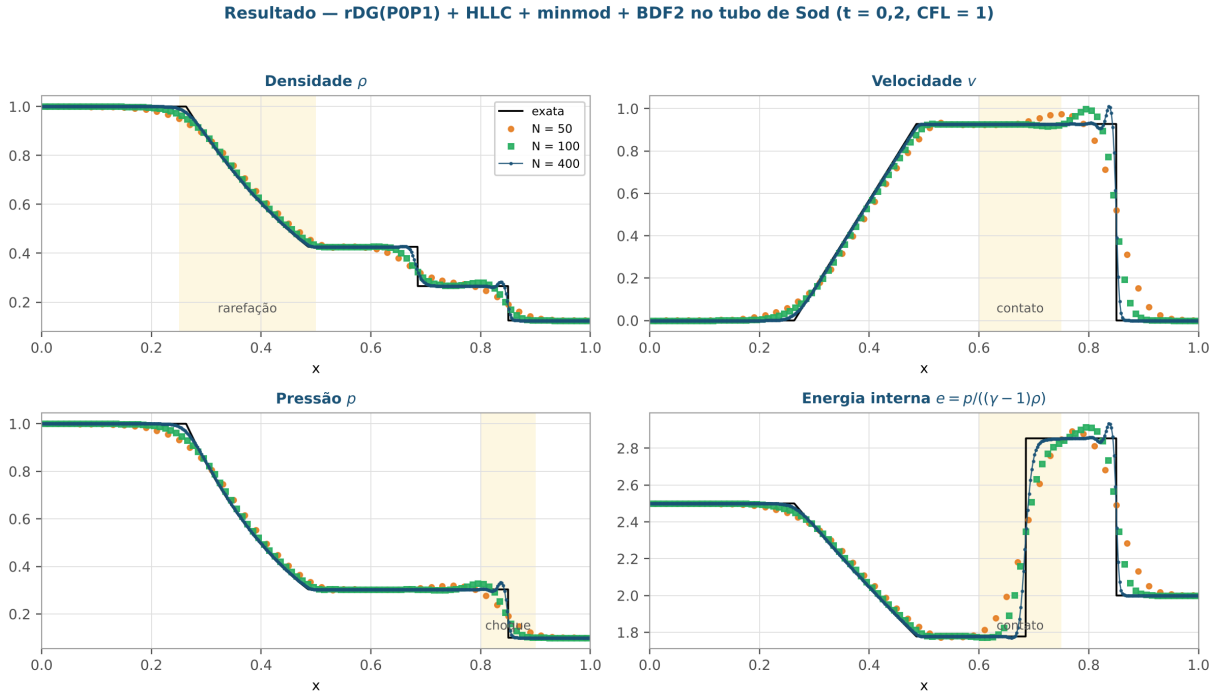


Figura 6: Resultado do rDG(P0P1) + HLLC + minmod + BDF2 para o problema de Sod ($t = 0{,}2$, CFL = 1) com três resoluções, nas quatro variáveis de interesse. A convergência com o refino da malha é visível em todas as estruturas: rarefação (esquerda), contato (centro) e choque (direita). O choque ocupa duas ou três células em qualquer malha; o contato é a estrutura mais borrada, como esperado; a energia interna, que o salta com amplitude grande, é a variável em que isso mais aparece. O pequeno sobressalto de velocidade e pressão logo atrás do choque é o erro de partida do BDF2 no instante inicial, quando o salto ainda ocupa uma célula.

9.1 Convergência com a malha

Olhar os perfis convence; medir o erro convence mais. A Tabela 1 e a Figura 7 mostram o erro L^1 da densidade, $E_N = \Delta x \sum_i |\bar{\rho}_i - \bar{\rho}_i^{\text{exata}}|$, para o método sem reconstrução (P0, primeira ordem) e com reconstrução minmod (rDG(P0P1)), em dois problemas: uma onda de densidade suave e periódica, $\rho_0(x) = 1 + 0{,}2\,\sin(2\pi x)$ com $v = 1$ e $p = 1$, transportada por uma volta inteira, e o tubo de Sod.

Três lições saem dos números. No problema suave, o P0 reduz o erro por um fator que tende a 2 ao dobrar N (primeira ordem), enquanto o rDG(P0P1) com minmod aproxima-se do fator 4 (segunda ordem); sem limitador o fator 4 é atingido desde as malhas grossas, porque o minmod reduz a ordem nos extremos da senoide. No problema de Sod, nenhum dos dois chega à sua ordem nominal, pois os saltos dominam o erro: o P0 fica em torno de $N^{-0{,}7}$ e o rDG(P0P1) em torno de $N^{-0{,}9}$. Ainda assim, o erro do rDG(P0P1) é cerca de três vezes menor em todas as malhas; a vantagem cresce com N: a reconstrução paga sempre, mesmo quando a solução tem choques.

10. Resumo e perspectivas

10.1 O que aprendemos

1. **Leis de conservação** descrevem o transporte de quantidades físicas; o **fluxo** $f(u)$ mede quanto passa por cada ponto. A forma integral continua válida nos choques; é dela que o método parte.

Tabela 1: Erro L^1 da densidade e razão entre erros ao dobrar N (ordem = $\log_2$ da razão). Onda de densidade em $t = 1$ e Sod em $t = 0{,}2$, ambos com CFL = 1.

	onda de densidade (suave)		Sod (com saltos)	
N	P0	rDG(P0P1)	P0	rDG(P0P1)
20	$7{,}97 \times 10^{-2}$	$1{,}98 \times 10^{-2}$	–	–
40	$4{,}96 \times 10^{-2}$ (1,6)	$8{,}60 \times 10^{-3}$ (2,3)	–	–
50	–	–	$3{,}05 \times 10^{-2}$	$1{,}71 \times 10^{-2}$
80	$2{,}79 \times 10^{-2}$ (1,8)	$2{,}55 \times 10^{-3}$ (3,4)	–	–
100	–	–	$2{,}10 \times 10^{-2}$ (1,5)	$9{,}94 \times 10^{-3}$ (1,7)
160	$1{,}48 \times 10^{-2}$ (1,9)	$7{,}29 \times 10^{-4}$ (3,5)	–	–
200	–	–	$1{,}35 \times 10^{-2}$ (1,6)	$5{,}68 \times 10^{-3}$ (1,8)
320	$7{,}62 \times 10^{-3}$ (1,9)	$1{,}98 \times 10^{-4}$ (3,7)	–	–
400	–	–	$8{,}45 \times 10^{-3}$ (1,6)	$3{,}06 \times 10^{-3}$ (1,9)
800	–	–	$5{,}24 \times 10^{-3}$ (1,6)	$1{,}60 \times 10^{-3}$ (1,9)

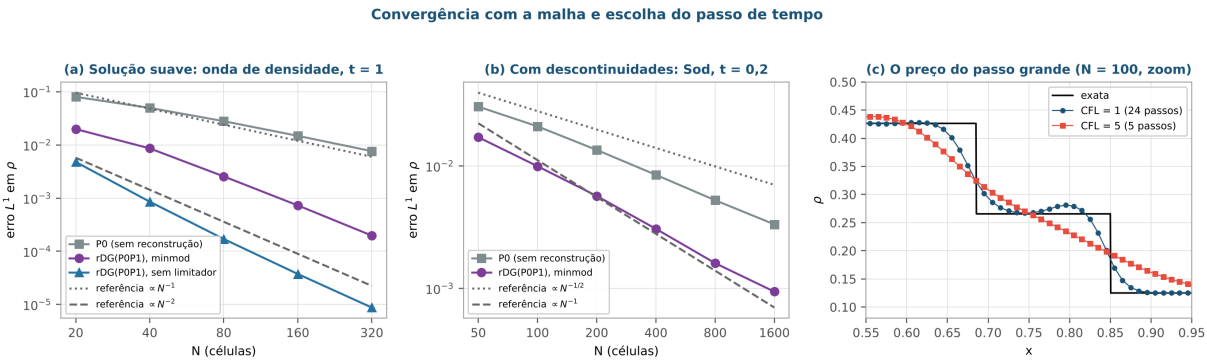


Figura 7: Convergência com a malha e efeito do passo de tempo. (a) Onda de densidade suave: P0 segue a referência de primeira ordem, o rDG(P0P1) com minmod aproxima-se da segunda ordem e sem limitador a atinge. (b) Tubo de Sod: as descontinuidades reduzem a ordem observada dos dois métodos, mas a reconstrução mantém o erro cerca de três vezes menor. (c) O mesmo problema com $N = 100$ e dois passos de tempo: com CFL = 5 o método continua estável, porém o contato e o choque ficam espalhados por muito mais células. Estabilidade incondicional não é precisão incondicional.

2. **Volumes finitos** discretizam a forma integral: a média celular muda conforme os fluxos nas interfaces. O que sai de uma célula entra na vizinha, logo o método é conservativo e os choques andam na velocidade certa.
3. **rDG(P0P1)** ganha segunda ordem ao reconstruir uma reta (P1) a partir das médias (P0), em vez de evoluir incógnitas adicionais como no DG puro. O coeficiente $\hat{u}_i$ é a variação do centro até a face.
4. O **minmod** limita as inclinações para evitar oscilações, degenerando para P0 perto de descontinuidades. É um esquema TVD: nenhum extremo novo é criado.
5. O **HLLC** resolve os problemas de Riemann locais nas interfaces, capturando corretamente choques, rarefações e descontinuidades de contato. O fluxo é o da região do diagrama de ondas em que a interface cai.
6. O **BDF2** integra no tempo de forma implícita e A-estável, permitindo passos de tempo muito maiores que métodos explícitos e viabilizando a simulação de problemas com escalas de tempo díspares. O sistema não linear de cada passo pede um Newton de verdade (JFNK); o passo grande é estável, mas borra as ondas rápidas.

10.2 Para onde ir

O framework rDG admite extensões naturais:

- **rDG(P0P2)**: reconstrução quadrática → terceira ordem;
- **rDG(P1P2)**: evolui $\bar{u}$ e $\hat{u}$ via DG, reconstrói o termo quadrático;
- **2D/3D**: a formulação DG se adapta naturalmente a triângulos/tetraedros;
- Limitadores mais sofisticados (WENO, Barth-Jespersen) e Riemann solvers (HLLC com correções entrópicas, Roe);
- Reconstrução nas variáveis primitivas ou características, que deixa o contato mais nítido; limitadores que garantem positividade de ρ e p, indispensáveis em escoamentos com vácuo ou com fases evanescentes;
- Integradores implícitos–explícitos (IMEX), que tratam implicitamente só a parte acústica e mantêm a convecção explícita, combinando o passo grande do BDF2 com a nitidez do explícito.

Documento gerado automaticamente com LaTeX e Python (NumPy + SciPy + Matplotlib).
Código-fonte completo disponível como material complementar: `scripts/rdg_p0p1.py` e `scripts/make_figs.sh`.